

Teoría de la Computabilidad

Módulo 12: El problema de la detención - Reducibilidad

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Problema de la Detención de MT

Un problema de decisión de especial importancia es el “problema de detención de la máquina de Turing” (*Halting Problem*)

Problema de Detención de MT (The Halting Problem): Dada una MT T y una cadena α , ¿existe un algoritmo para decidir si T se detendrá comenzando en el estado inicial con α en la cinta?

¡Problema Insoluble!

Problema de la Detención de MT



Algoritmo DetenciónMT

Dato de Entrada: T, α

Dato de Salida: Estado

Comienzo

.....

Devolver Estado con valor
“se detiene” o “cicla”

Fin

Turing ha demostrado que este
algoritmo NO EXISTE

Intuición de la demostración de Turing

Supongamos que el problema es soluble, y el procedimiento

EjecYSiempreParar($P, E, \underline{Res}$)

recibe una MT P y una entrada E , y simula P sobre E .

Si P se detiene con la entrada E , devuelve “Sí”.
Si P entra en ciclo infinito, retorna “No”.

P es una descripción codificada de una Máquina de Turing

Intuición de la demostración de Turing

En particular, si la entrada es la misma descripción, es válido invocar a:

EjecYSiempreParar($P, P, \underline{Res}$)

En este caso nos dice si la MT P se detiene o no comenzando con la cadena P en la cinta.

Consideremos el siguiente algoritmo, que cicla infinitamente en un caso en particular...

Intuición de la demostración de Turing

Procedure Diagonal(X):

Repetir

EjecYSiempreParar($X, X, \underline{Res}$)

Hasta $Res = \text{No}$

El procedimiento Diagonal para sssi
EjecYSiempreParar responde “No” (X cicla)

El procedimiento Diagonal cicla infinitamente
sssi EjecYSiempreParar responde “Sí” (X se detiene)

Intuición de la demostración de Turing

```

Procedure Diagonal(X):
  Repetir
    EjecYSiempreParar(X,X,Res)
  Hasta Res=No
    
```

Diagonal **se detiene** si X **cicla**

Diagonal **cicla** infinitamente si X **se detiene**

Intuición de la demostración de Turing

¿Qué pasa si ejecutamos **Diagonal(Diagonal)**?

```

Procedure Diagonal(Diagonal):
  Repetir
    EjecYSiempreParar(Diagonal,Diagonal, Res)
  Hasta Res=No
    
```

El procedimiento Diagonal para sssi EjecYSiempreParar responde “No” (Diagonal **cicla**)

El procedimiento Diagonal **cicla** infinitamente sssi EjecYSiempreParar responde “S” (Diagonal **se detiene**)

Intuición de la demostración de Turing

¿Qué pasa si ejecutamos **Diagonal(Diagonal)**?

```

Procedure Diagonal(Diagonal):
  Repetir
    EjecYSiempreParar(Diagonal,Diagonal, Res)
  Hasta Res=No
    
```

Diagonal **se detiene** si Diagonal **cicla**

Diagonal **cicla** si Diagonal **se detiene**

Esto es absurdo, y proviene de asumir que existe el algoritmo para resolver el problema de la detención.

Luego el problema de la detención es insoluble.

Formalizando la intuición de Turing

Sea el lenguaje

$H = \{ \langle M, w \rangle \mid \text{la máquina de Turing } M \text{ se detiene sobre el input } w \}$.

H es **recursivo enumerable** (pues la MUT justamente se detiene cuando w es aceptado por M).

¿Será H **recursivo**? Si lo fuera, existiría una MT M_0 que decide si M se detiene al procesar w (esta máquina M_0 resolvería el problema de la detención).

Si H es **recursivo**, ent. el lenguaje

$H_1 = \{ \langle M \rangle \mid \text{la MT } M \text{ para sobre el input } M \}$ también lo es.

Formalizando la Intuición de Turing

Si

$H_1 = \{ \langle M \rangle \mid \text{la máquina de Turing } M \text{ para sobre el input } M \}$

es **recursivo** también lo es su complemento CH_1 (por teorema antes visto):

$CH_1 = \{ w \mid \text{o bien } w \text{ no codifica una MT, o bien } w = \text{Cod}(M) \text{ tal que } w \notin L(M) \}$.

Casualmente, el lenguaje CH_1 es análogo a nuestro procedimiento Diagonal, y el último paso de nuestra prueba. ... ¿Puede CH_1 ser **recursivo**?

Formalizando la Intuición de Turing

Supongamos que hubiera solución para el problema de la detención

Puedo definir un lenguaje **recursivo**
 $H = \{ \langle M, w \rangle \mid M \text{ para con input } w \}$.

El lenguaje $H_1 \subset H$, con
 $H_1 = \{ \langle M \rangle \mid M \text{ para con input } M \}$
 también sería **recursivo**...

El complemento de H_1 es $CH_1 = \Sigma^* - H_1$.
 $CH_1 = \{ w \mid \text{o bien } w \text{ no codifica una MT, o bien } w = \text{Cod}(M) \text{ tal que } w \notin L(M) \}$.
 Si H_1 es **recursivo**, entonces CH_1 debería serlo (por propiedad)

Formalizando la Intuición de Turing

$CH_1 = \{w \mid w \text{ no codifica una MT, o bien } w = \text{Cod}(M) \text{ tal que la MT } M \text{ no se detiene con input } w\}$.

¿Puede CH_1 ser recursivo? Veamos que CH_1 no es recursivo, ¡ni siquiera r.e.!

Supongamos que M^* es una MT que hace que CH_1 sea r.e.

M^* para si $w \in CH_1$,

M^* no necesariamente para si $w \notin CH_1$

Entonces ¿ $\text{Cod}(M^*) \in CH_1$?

Formalizando la Intuición de Turing

¿ $\text{Cod}(M^*) \in CH_1$?

• Por def. de CH_1 ,

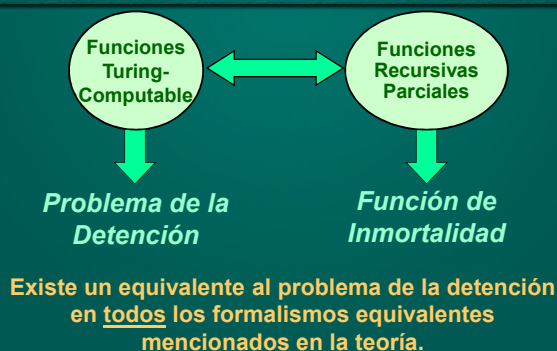
$\text{Cod}(M^*) \in CH_1$ sssi M^* no acepta $\text{Cod}(M^*)$.

• Por def. de r.e., si M^* la aplicamos a $\text{Cod}(M^*)$, se tiene que

$\text{Cod}(M^*) \in CH_1$ sssi M^* acepta $\text{Cod}(M^*)$.

Este absurdo partió de suponer que M_0 existe. Luego M_0 no existe, y el problema de la detención es insoluble.

Funciones Recursivas Parciales y Maq. de Turing



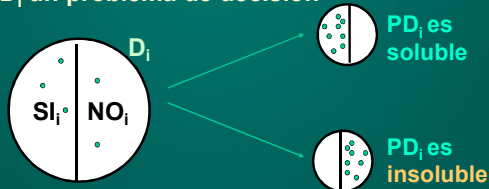
Reducibilidad

- Para probar que un problema de decisión es insoluble se debe probar que no existe algoritmo para realizar una tarea particular.
- Es importante establecer relaciones entre problemas de decisión, que nos permitan “reducir” un problema a otro conocido.
- Halting Problem = punto de referencia de interés (pues es insoluble)



Conceptualización de PDs

Sea PD_i un problema de decisión



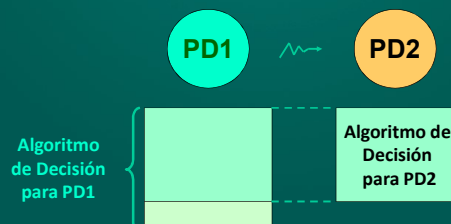
D_i = dominio problema de decisión $PD_i = SI_i \cup NO_i$

• = instancia del problema PD_i

Ej: $PD =$ dado un nro. N primo, N natural, existe un nro. $M > N$ tal que M es primo.

Reducibilidad: $PD_1 \sim PD_2$

Dados dos problemas de decisión PD_1 y PD_2 , diremos que PD_1 se reduce a PD_2 si un algoritmo usado para solucionar PD_2 puede usarse para construir la solución para PD_1 (notación $PD_1 \sim PD_2$).



Ejemplo de reducibilidad

N/D
¿es irreducible?

Simplifico N/D lo más posible, obtengo N'/D'

D'
¿D' divide a N'?

Ej: 105 / 30 ¿es irreducible? 105 / 30 = 7/2 ¿2 divide a 7?

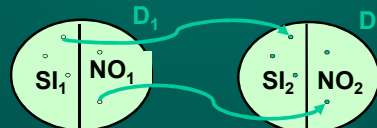
Algoritmo Irreduc1
D.Entrada: N, D
D.Salida: Eslrreduc.
Comienzo
MCD ← ObtenerMCD(N,D)
N' ← N / MCD
D' ← D / MCD
Eslrreduc ← N'//D'=0
Fin

Algoritmo Irreduc2
D.Entrada: N', D'
D.Salida: Eslrreduc.
Comienzo
Eslrreduc ← N'//D'=0
Fin

Reducibilidad: PD1 $\rightsquigarrow$ PD2

Formalmente:

PD1 $\in$ **PD2** si $\exists$ MT **M** que toma como entrada una codificación de una instancia $I_1 \in D_1$ de **PD1** y devuelve $I_2 \in D_2$, instancia de **PD2**, tq. si $I_1 \in SI_1$ ent. $I_2 \in SI_2$ y si $I_1 \in NO_1$ ent. $I_2 \in NO_2$



Teorema de Reducibilidad

Teorema: Sean **PD1** y **PD2** prob. de decisión:

- (1) Si **PD1** $\rightsquigarrow$ **PD2**, y **PD2** es soluble, ent. **PD1** también es soluble.
- (2) Si **PD1** $\rightsquigarrow$ **PD2**, y **PD1** es insoluble, ent. **PD2** también es insoluble.

Dem (1): si **PD1** $\rightsquigarrow$ **PD2**, ent. $I_1 \in SI_1$ sssi $I_2 \in SI_2$ y $\exists$ MT T_1 que toma como entrada una codificación $I_1 \in D_1$ y devuelve $I_2 \in D_2$. Por hip., **PD2** es soluble. Luego $\exists$ MT T_2 tq. dada $I_2 \in D_2$ decide $I_2 \in SI_2$ ó $I_2 \in NO_2$.

Pero ent. combinando ambas máquinas T_1 y T_2 podemos saber si $I_1 \in SI_1$ ó $I_1 \in NO_1$

Teorema de Reducibilidad

Teorema: Sean **PD1** y **PD2** prob. de decisión:

- (1) Si **PD1** $\rightsquigarrow$ **PD2**, y **PD2** es soluble, ent. **PD1** también es soluble.
- (2) Si **PD1** $\rightsquigarrow$ **PD2**, y **PD1** es insoluble, ent. **PD2** también es insoluble.

Dem: (2) sabemos por hipótesis que a) **PD1** $\in$ **PD2**, y b) **PD1** es insoluble.

Sup. por absurdo **PD2** es soluble. Pero ent. por (1) **PD1** también debería ser soluble, y no es así (la hipótesis dice lo contrario!). Luego **PD2** no es soluble, CQD.

Corolario del Teorema Anterior

Corolario: Sea **PDet** = problema de la detención de la Máquina de Turing, y sea **P_i** un problema de decisión arbitrario.

Entonces si **PDet** $\rightsquigarrow$ **P_i**, entonces el problema **P_i** es insoluble.

Este resultado será de utilidad para resolver distintos problemas de decisión.

Problema de la Detención de MT

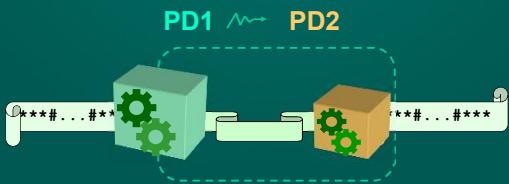
Un problema de decisión de especial importancia es el “problema de detención de la máquina de Turing” (*Halting Problem*)

Problema de Detención de MT (The Halting Problem): Dada una MT **T** y una cadena α , ¿existe un algoritmo para decidir si **T** se detendrá comenzando en el estado inicial con α en la cinta?

¡Problema Insoluble!

Reducibilidad: $PD1 \in PD2$ - Repaso

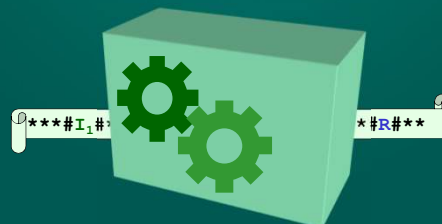
Dados dos problemas de decisión $PD1$ y $PD2$, diremos que $PD1$ se reduce a $PD2$ si un algoritmo usado para solucionar $PD2$ puede usarse para construir la solución para $PD1$ (notación $PD1 \rightsquigarrow PD2$).



Reducibilidad: $PD1 \rightsquigarrow PD2$ - Repaso

Formalmente:

$PD1 \in PD2$ si $\exists$ MT M que toma como entrada una codificación de una instancia $I_1 \in D_1$ de $PD1$ y devuelve $I_2 \in D_2$, instancia de $PD2$, tq. si $I_1 \in SI_1$ ent. $I_2 \in SI_2$ y si $I_1 \in NO_1$ ent. $I_2 \in NO_2$



Teorema de Reducibilidad - Repaso

Teorema: Sean $PD1$ y $PD2$ prob. de decisión:

- (1) Si $PD1 \in PD2$, y $PD2$ es soluble, ent. $PD1$ también es soluble.
- (2) Si $PD1 \in PD2$, y $PD1$ es insoluble, ent. $PD2$ también es insoluble.



Corolario del Teorema Anterior

Corolario: Sea PD_{Det} = problema de la detención de la Máquina de Turing, y sea P_i un problema de decisión arbitrario.

Entonces si $PD_{\text{Det}} \in P_i$, entonces el problema P_i es insoluble.

Este resultado será de utilidad para resolver distintos problemas de decisión.

Estrategia de demostración por reducibilidad

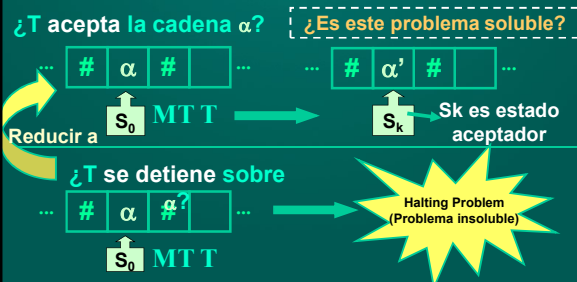
Para demostrar que un problema es insoluble:

1. **Suponer** que el problema es soluble.
2. Tomar un problema que ya se conozca, o pueda demostrarse, insoluble, para plantear la reducción.
3. Es importante recordar que debe reducirse el problema elegido en el punto 2 al problema que supusimos soluble (*demonstración por el absurdo*)
4. Demostrar la reducción.

No olvidar la conclusión de la demostración.

Problema: Aceptación de una cadena α

P_{accept} : Dada una MT T , ¿existe un procedimiento efectivo para decidir si T acepta una determinada cadena α ?



Caso de estudio: P_{accept}

Para demostrar que un problema es insoluble:

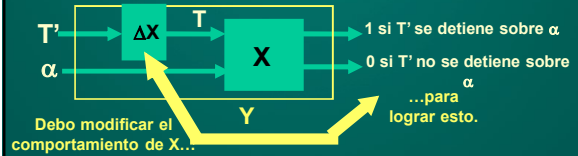
1. Suponer que el problema es soluble.



Supongamos que P_{accept} es soluble. Entonces existe un procedimiento efectivo (o MUT) X que resuelve P_{accept} : dado un par (T, α) , retorna un 1 si T acepta α , y un 0 en caso contrario.

Caso de estudio: P_{accept}

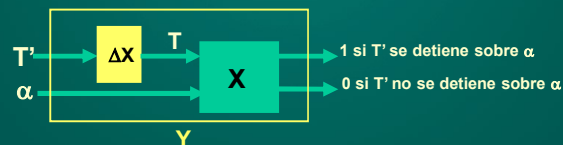
- 2 y 3. Tomar un problema que ya se conozca, o pueda demostrarse, insoluble, para plantear la reducción (ej. PDet). La reducción nos debe llevar a un absurdo.



A partir de X construiremos una nueva máquina Y . Esta máquina recibe como entrada (T', α) . La cadena de entrada es la misma, pero debemos modificar T' de manera que las respuestas de X correspondan al "Halting problem". Eso lo podemos hacer a través de una modificación ΔX

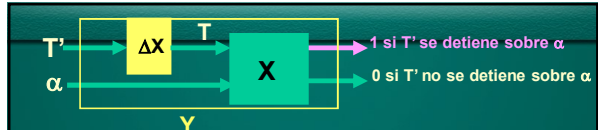
Caso de estudio: P_{accept}

- 2 y 3. Tomar un problema que ya se conozca, o pueda demostrarse, insoluble, para plantear la reducción (ej. PDet). La reducción nos debe llevar a un absurdo.



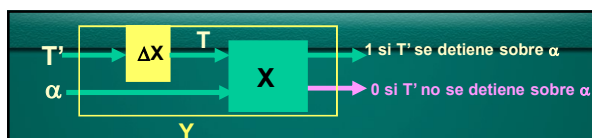
¿qué definición podemos adoptar para ΔX ?

Tomar como entrada una máquina T' , modificar T' dejando las quintuplas tal como estaban, pero indicando que todos los estados de T' son aceptadores. La máquina resultante obtenida la llamaremos T , y es la entrada para la máquina X que ya teníamos.



Combinando X y ΔX tenemos entonces una máquina universal Y que tiene el sgte. comportamiento:

1. La máquina universal Y recibe como entrada a un par (T', α) . La máquina Y utiliza ΔX a partir de T' para construir T , usando la estrategia antes indicada.
2. El par (T, α) ingresa a la máquina universal X .
3. Si X responde 1, entonces T acepta α . Pero entonces T' se detiene sobre α (pues T se comporta como T' , solo que siempre que T se detiene es porque T acepta). Luego si X devuelve 1, es porque T' se detiene sobre α .



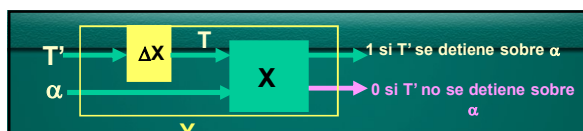
Combinando X y ΔX tenemos entonces una máquina universal Y que tiene el sgte. comportamiento:

4. Si X responde 0, entonces T no acepta α . Pero entonces T' se no detiene sobre α (pues T se comporta como T' , solo que siempre que T se detiene es porque T acepta). Luego si X devuelve 0, es porque T' no se detiene sobre α .

Conclusión:

Si X devuelve 1, es porque T' se detiene sobre α .

Si X devuelve 0, es porque T' no se detiene sobre α .



Si X devuelve 1, es porque T' se detiene sobre α .

Si X devuelve 0, es porque T' no se detiene sobre α .

La máquina Y retorna 1 (T' se detiene sobre α) si X devuelve 1
La máquina Y retorna 0 (T' se no detiene sobre α) si X devuelve 0
Luego Y resuelve PDet, el problema de la detención (ABSURDO).

Por hipótesis sabemos que PDet es insoluble, por lo que la solución encontrada por la máquina Y no puede existir.

Esto implica que alguna de sus componentes no existe:
o bien no existe ΔX , o bien no existe X .

Pero ΔX existe por construcción.
Luego no existe X . Luego P_{accept} no es soluble, C.Q.D.